About Software Deployment Standard - (SDPS)

About Software Deployment Standard - (SDPS)

Canonical Definition

SDPS — Software Deployment Standard is the Parent Standard within SLGA™ that governs how authorized software releases are prepared, approved for deployment, introduced into defined operational environments, associated with identifiable production versions, monitored for deployment-state changes, and subjected to rollback or recovery when deployment conditions cannot be maintained.

SDPS establishes software deployment as a governed operational transition

rather than a technical installation event. It requires deployment authority, target environment, execution conditions, production identity, operational verification, and recovery obligations to remain distinguishable, traceable, and evidence-supported.

Its governance logic applies to human-operated deployments, automated

delivery pipelines, autonomous AI agents, distributed infrastructure, cloud platforms, embedded software, and robotic systems.

• What It Is

SDPS is a methodological framework for governing the transition of software from an authorized release into an operational environment.

It establishes five interconnected governance mechanisms:

Deployment Planning: Defining the deployment purpose, target

environment, execution strategy, prerequisites, operational constraints, and recovery conditions. 2. Deployment Authorization: Establishing legitimate permission to deploy a specific release into a specific environment under defined conditions. 3. Production Version Control: Identifying and verifying which software artifacts and versions are actually present and active in operational environments. 4. Deployment State Monitoring: Observing and evaluating whether the actual deployment state satisfies applicable operational conditions. 5. Rollback & Recovery: Governing corrective actions when deployment failures, unacceptable conditions, or operational disruptions require intervention.

Together, these mechanisms establish a traceable relationship between what

was authorized, what was executed, what is actually operating, and what must happen when the resulting state is unacceptable.

• What It Is Not

SDPS is not a software deployment tool, CI/CD platform, orchestration engine, or technical installation procedure.

It does not prescribe a particular cloud provider, programming language, deployment framework, infrastructure architecture, or automation technology.

It does not replace software release governance. A release may be authorized for publication without being authorized for deployment into a particular production environment.

It does not assume that successful execution means successful deployment. A pipeline may finish without errors while an incorrect artifact, incompatible configuration, or degraded service remains in production.

It does not treat monitoring as proof of complete operational visibility, nor does it treat rollback execution as proof of successful recovery.

SDPS also does not grant authority to human operators or autonomous agents. It governs the application and verification of legitimate deployment authority established through the relevant governance mechanisms.

• Why It Is Important

Modern software deployment increasingly occurs through automated pipelines, distributed infrastructure, continuous delivery systems, and autonomous AI agents.

These systems can introduce software changes rapidly, sometimes across thousands of operational instances.

However, technical execution capability does not establish whether a deployment is legitimate, whether the intended software was actually installed, or whether the resulting environment remains operationally acceptable.

A software release may be fully approved while its deployment into a particular environment is unauthorized.

A deployment pipeline may report success while production runs an artifact different from the authorized release.

A monitoring system may show normal service availability while significant parts of the deployment remain unobserved.

An automated rollback may restore an earlier software binary while leaving databases, dependencies, or infrastructure in an incompatible state.

SDPS addresses these gaps by requiring independent governance over

deployment decisions, execution, production identity, operational verification, and recovery.

Its importance increases as software systems become more autonomous and the

consequences of deployment errors extend beyond software functionality into financial systems, infrastructure, industrial operations, and physical environments.

The ability to deploy software must never be confused with the authority to deploy it or with evidence that the resulting operational state is acceptable.

Position in Architecture Governance Flow

SDPS occupies the eighth Parent Standard position within SLGA™ — Software Lifecycle Governance Architecture.

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SDPS follows:

SRLS — Software Release Standard

SRLS governs release candidates, readiness, authorization, records, and

withdrawal.

SDPS receives the relevant release identity and authorization references

but independently governs whether, where, and under what conditions the release may enter an operational environment.

SDPS precedes:

SCMS — Software Change Management Standard

SCMS governs change requests, impact assessment, coordination, lifecycle

re-entry, and change closure.

SDPS provides the deployment evidence and operational state references

needed to understand how software changes affect deployed environments.

SDPS also supports Software Retirement and Software Lifecycle Control by

preserving production version identities, deployment histories, operational states, and recovery references.

Standard Governance Flow

Authorized Software Release → Deployment Purpose → Target Environment →

Deployment Planning → Deployment Prerequisites → Deployment Authority → Deployment Authorization → Deployment Execution → Production Version Identification → Deployment State Monitoring → Operational Verification → Continue / Restrict / Intervene / Roll Back / Recover → Deployment Governance Record

This flow distinguishes the intended deployment from the permitted

deployment, the permitted deployment from its actual execution, and the executed deployment from the operational state ultimately established.

The five Core Modules implement this governance structure:

DPM — Deployment Planning Module

Establishes the deployment plan, target environment, execution strategy,

prerequisites, and verification and recovery conditions.

DAM — Deployment Authorization Module

Determines whether the deployment is legitimately permitted for the specified release, environment, actor, scope, and conditions.

PVCM — Production Version Control Module

Identifies the software versions and artifacts actually present and

operating in production.

DSMM — Deployment State Monitoring Module

Observes deployment conditions, evaluates operational evidence, and

identifies deviations requiring reassessment or intervention.

RRVM — Rollback & Recovery Module

Governs rollback feasibility, corrective execution, recovery verification,

and residual operational conditions.

These modules form a connected governance sequence while preserving

separate responsibilities and outputs.

Practical Applications

Enterprise Software Deployment

Establishing deployment authorization, production version traceability,

operational verification, and recovery conditions across business-critical applications.

Continuous Integration and Continuous Delivery

Distinguishing automated pipeline execution from legitimate deployment

authorization and verified production state.

Autonomous AI Software Agents

Ensuring that agents capable of deploying software remain within defined authority, tool-access boundaries, target environments, and operational constraints.

Cloud and Distributed Infrastructure

Maintaining visibility over software versions, staged rollouts, regional

deployments, containerized services, and mixed-version production environments.

Critical Infrastructure and Industrial Systems

Governing software updates where deployment failures may affect essential services, equipment, industrial processes, or physical operations.

Robotics and Embedded Systems

Verifying deployment authority, software identity, activation conditions,

and recovery requirements when software changes influence physical system behavior.

Emergency Security Deployment

Supporting urgent deployment decisions while preserving authorization,

scope, exception handling, evidence, and recovery obligations.

Governance Outputs

SDPS establishes the following principal governance output:

Software Deployment Governance Profile

This profile provides a structured representation of the deployment's

identity, authorization, execution, production state, operational verification, and applicable recovery conditions.

Supporting outputs include:

• Software Deployment Plan Profile — DPM

• Software Deployment Authorization Profile — DAM

• Software Production Version Profile — PVCM

• Software Deployment State Profile — DSMM

• Software Deployment Recovery Profile — RRVM

Additional governance records may include deployment execution evidence, target environment references, production artifact identity, monitoring results, operational exceptions, rollback decisions, recovery verification, and historical deployment states.

These outputs support auditability, operational reassessment,

accountability references, lifecycle continuity, and evidence-based governance decisions.

Core Governance Distinctions

Release Authorized ≠ Deployment Authorized.

Permission to publish or make software available does not automatically authorize its introduction into a particular operational environment.

Technical Deployment Capability ≠ Legitimate Deployment Authority.

Access to infrastructure or deployment tools does not independently establish permission to execute deployment actions.

Deployment Executed ≠ Deployment Successfully Established.

Completion of technical deployment actions does not prove that the intended operational state was achieved.

Software Version Label ≠ Verified Production Artifact Identity.

A displayed version number does not independently prove which artifact is actually running.

Software Running ≠ Software Operating Within Authorized Conditions.

A functioning application may still violate configuration, security, operational, or authorization requirements.

Deployment Monitoring Active ≠ Deployment State Fully Observable.

Monitoring coverage and evidence sufficiency must be evaluated rather than assumed.

Agent Task Authority ≠ Authority for Every Tool Action.

An autonomous agent's assigned deployment objective does not authorize every infrastructure modification or downstream action it could technically perform.

Rollback Executed ≠ Operational Recovery Verified.

Restoring a previous software version does not necessarily restore compatible data, dependencies, infrastructure, or service functionality.

Release Withdrawal ≠ Deployment Rollback.

Removing a release from distribution does not automatically remove or reverse existing operational deployments.

Deployment Completed ≠ Deployment Governance No Longer Required.

Production identity, operational conditions, monitoring, recovery, and

lifecycle continuity remain relevant after deployment execution ends.

Canonical Closing Statement

Software deployment is not merely the movement of code into production. It is the transfer of an authorized software release into a real operational environment where technical actions produce operational consequences.

A release can be legitimate while its deployment is unauthorized.

A deployment can execute successfully while the wrong software artifact is running.

An automated pipeline can report success while the operational environment remains degraded.

An autonomous agent can possess deployment capabilities without possessing legitimate authority over the actions it performs.

And a rollback can complete technically without restoring an acceptable operational state.

SDPS establishes the governance mechanisms required to distinguish these

conditions.

It connects deployment planning, legitimate authorization, production

version identity, operational monitoring, and recovery into one coherent methodological structure.

The purpose of SDPS is not simply to ensure that software reaches production. It is to ensure that software enters, operates, changes, and can be recovered within identifiable, authorized, evidence-supported, and verifiable operational boundaries.

Deployment capability determines what a system can execute. Deployment governance determines what it may execute, under which conditions, and how the resulting operational reality is verified.